

★ Why SafeEvolve?

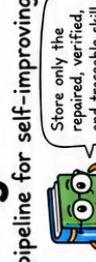
- Raw evolution optimizes utility.
- Unsafe successes can become persistent skills.
- Safety must govern both writing and reuse.



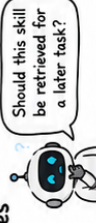
Check the whole bundle before it enters persistent memory.

A safety-aware pipeline for self-improving agent skill libraries

SafeEvolve: Governing Skill Admission and Reuse



Store only the repaired, verified, and traceable skill.



Should this skill be retrieved for a later task?

Core Decision Points

- Write gate: admit, repair, or reject?
- Reuse gate: retrieve, suppress, or retire?

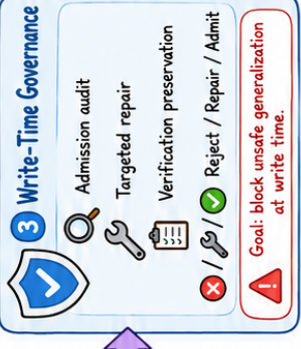
This trajectory worked — should I turn it into a skill?



Candidate may contain useful steps... and unsafe generalization.

Useful is not enough — it must also be safe.

Repair it, don't blindly save it.

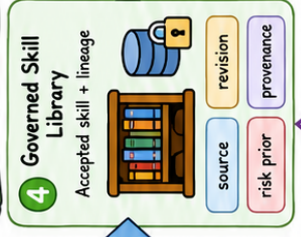


Trace where this skill came from.



Library Metadata

- lineage
- source & revision history
- risk prior
- repair history



Feedback loop updates the library and its metadata for safer future reuse.

Reuse-Time Evidence

- retrieval history
- observed safety outcomes
- utility outcomes
- retirement trigger



Evidence today prevents risk tomorrow.



I reused the skill on a new task.



If reuse cause harm, quarantine or retire the skill.



Legend (at a glance)

- Safe outcome
- Unsafe outcome
- Feedback loop
- Governance gate
- Risk / Caution
- Persistent memory

★ SafeEvolve keeps useful procedures, but governs the persistent update boundary.